DataScientest / Liora

Classification of Rakuten E-commerce Products

Rakuten France Multimodal Product Data Classification (ENS Challenge Data 35; also on Kaggle)

Prepared by
Sandeep Grover, Jonathan Vints, Thomas Maisch
Data Scientist project team

Date: 20 July 2026

Contents

1. Introduction

1.1 Context

Cataloguing products from their text and image is a core problem for any e-commerce marketplace: the correct category drives search, filtering, recommendation and merchandising. On Rakuten France, millions of third-party sellers upload products with a short title (designation), an optional free-text description, and a single product image. Assigning each listing to the correct product-type code by hand does not scale, and seller-supplied labels are unreliable, because titles are free-form and multilingual, many products have no description, and some are mislabelled.

1.2 Objective

The task is to predict the product-type code (prdtypecode) of each listing from its textual and image data. This is the public Rakuten France Multimodal Product Data Classification challenge [1] (ENS Challenge Data 35, also distributed on Kaggle). The full catalogue spans thousands of classes; the benchmark exposes a curated subset of 27 product-type codes covering the main verticals.

1.3 Project framing

This is a team project, delivered by the three of us. This report, the exploration, data-visualization and pre-processing report, covers the business framing, the data exploration, and the transformation of the raw text into features. Two further team reports follow: a modelling report comparing classical and deep-learning models, and a final report adding the image branch and multimodal fusion.

2. Understanding the data

The data come from the Rakuten France challenge. Each listing provides four fields and one image, summarised in the box below.

> **Dataset:** Rakuten France Multimodal Product Data Classification (ENS Challenge Data 35; also on Kaggle).
> **Volume:** 84,916 labelled training listings; 13,812 unlabelled test listings. Text approximately 60 MB; images approximately 2.2 GB of 500 by 500 RGB JPEGs.
> **Fields:** designation (title, always present); description (optional, HTML-formatted); productid and imageid (image link).
> **Target:** prdtypecode, one of 27 product-type codes.

Because the categories are strongly imbalanced and multilingual, the exploration below quantifies four properties of the data, each of which drives a concrete pre-processing or modelling decision. Every statistical test in this report exists to justify one such decision.

3. Data exploration and visualization

3.1 Class imbalance

The 27 categories are heavily imbalanced: the largest class holds 10,209 listings and the smallest 764, a ratio of 13.4 to 1. A chi-square goodness-of-fit test against a uniform distribution gives a chi-square statistic of 36,570 ($p < 0.001$), so the imbalance is real, not sampling noise (Figure 1). Because the Rakuten challenge is scored on the weighted-F1 score, weighted-F1 is the primary metric; macro-F1 is also tracked, because the imbalance means rare-class performance must be watched. Class weighting and stratified splits are applied.

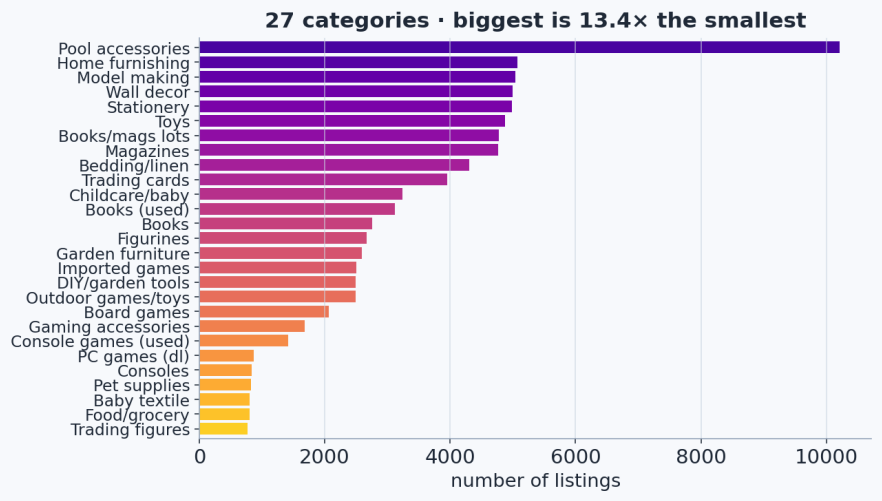

Figure 1. Distribution of the 27 product categories, showing the 13.4-to-1 imbalance.

3.2 Missing descriptions: a Missing-Not-At-Random signal

35.1% of listings have no description. Crucially, the missing rate is not uniform: it varies strongly across categories. A chi-square test of independence between description present or absent and the class gives a chi-square statistic of 43,758 and a Cramer's V of 0.72 [3] ($p < 0.001$), a strong association (Figure 2). The missingness is therefore Missing-Not-At-Random (MNAR) [2]: its very presence is informative. Rather than dropping these rows, which would discard about 35% of the data and hurt exactly the hardest categories, a binary desc_missing indicator is added so the model can use the absence as a feature.

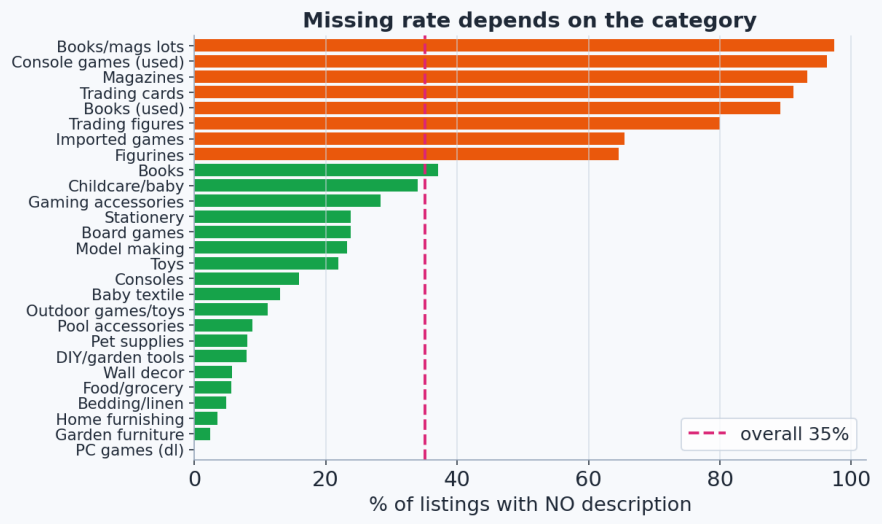

Figure 2. Missing-description rate per category, showing a strong category dependence (MNAR).

3.3 Text length varies by category

Total text length (title plus description) is heavily right-skewed: most listings are short, while a minority run to thousands of characters. Because the distribution is skewed, a non-parametric Kruskal-Wallis test [4] is used instead of a normal ANOVA; it confirms that length differs significantly across categories ($p < 0.001$, Figure 3). The designation and description lengths are therefore retained as numeric features.

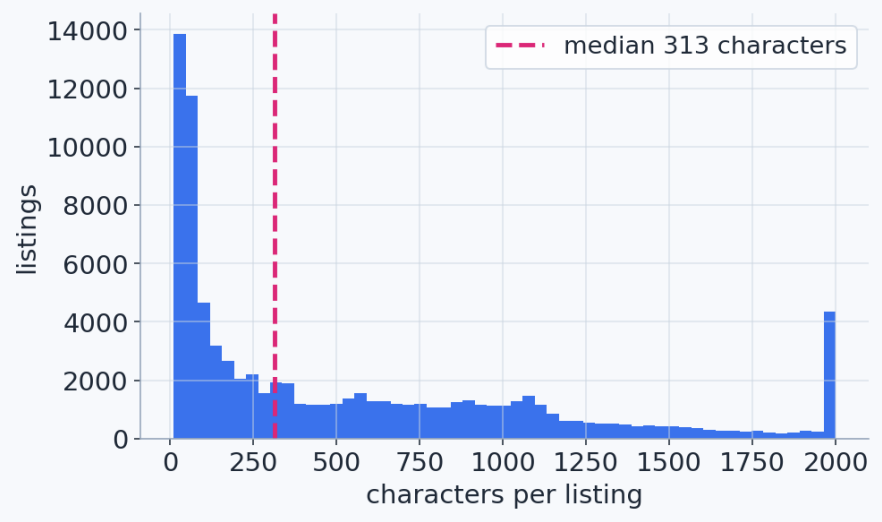

Figure 3. Distribution of text length per listing, with the median marked; note the heavy right tail.

3.4 Language distribution

The text is predominantly French but contains a genuine English and German tail, reflecting sellers from across Europe. Wilson 95% confidence intervals [5], which remain valid for the rare languages where the normal approximation breaks down, confirm that English and German are real sub-populations rather than noise (Figure 4). This favours a French or multilingual language model in the modelling stage, such as CamemBERT (a French RoBERTa), FlauBERT, or XLM-R.

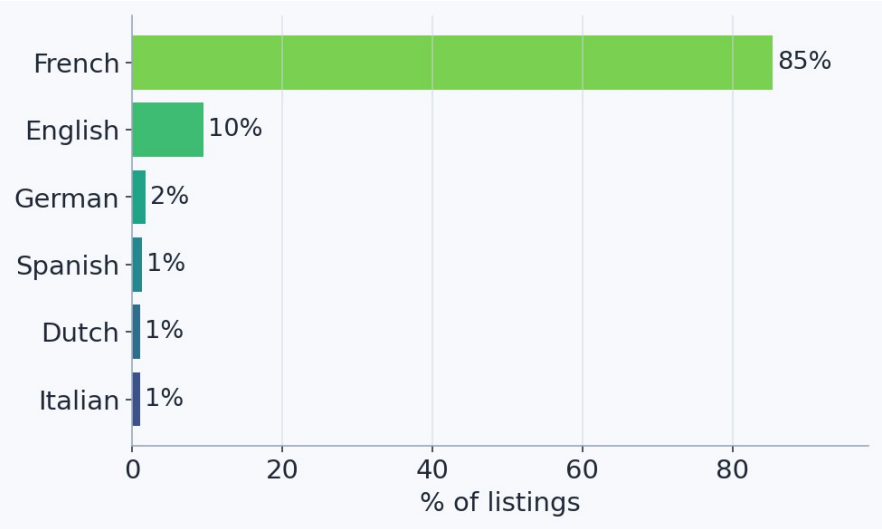

Figure 4. Language distribution across listings: predominantly French, with an English and German tail.

3.5 Word frequency

Counting how often each word occurs shows that the most frequent words are almost all stopwords, such as de, la, et, pour and les (Figure 5). These appear in nearly every listing and carry no category signal, which is precisely why TF-IDF [6] weights them down through the inverse-document-frequency term. At the other extreme, the least frequent words are one-off typos and serial numbers that appear only once. Both extremes are removed: rare tokens by the minimum-document-frequency threshold, and the long tail by capping the vocabulary at the 20,000 most useful terms; the discriminative words sit in between.

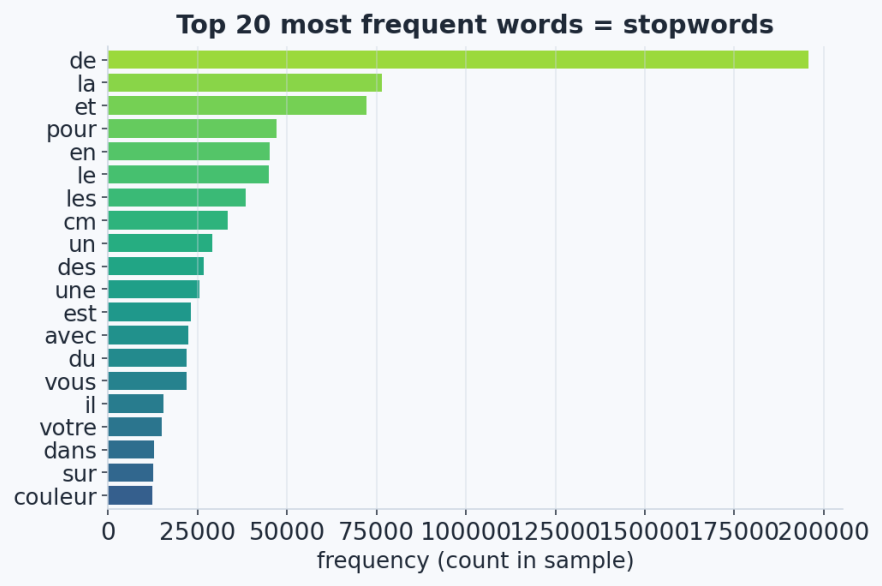

Figure 5. The 20 most frequent words in the corpus are all stopwords, motivating the inverse-document-frequency weighting in TF-IDF.

3.6 Summary of findings

Table 1. Exploratory findings and the pre-processing or modelling recommendation each one drives.

Finding	Key figures	Potential recommendation
Class imbalance	13.4:1 gap; chi-square = 36,570; $p < 0.001$	Weighted-F1 and macro-F1; class weighting; stratified split
Missing descriptions	35.1%; MNAR (chi-square = 43,758, V = 0.72; $p < 0.001$)	Add a desc_missing indicator; do not drop the rows
HTML in descriptions	Present in most non-empty descriptions	Strip with BeautifulSoup before TF-IDF
Text length by class	Kruskal-Wallis; $p < 0.001$	Add designation_len and description_len features
Multilingual content	French ~61%, English ~22%, German ~7%	French or multilingual model (CamemBERT, XLM-R)
Text to features	TF-IDF; 20,000 features; 80/20 split	Vectorise merged text; fit on the training set only

Note. Percentages for languages are Wilson 95%-CI point estimates from a labelled sample. Chi-square and Kruskal-Wallis statistics are computed on the full training set of 84,916 listings.

4. Pre-processing and feature engineering

4.1 Text cleaning (per listing, before the split)

The following operations act on each listing independently, so they carry no information between rows and are applied to the whole dataset before splitting. The HTML is stripped with BeautifulSoup [7], which decodes entities and inserts separators, so that a fragment such as rouge
bleu becomes rouge bleu rather than rougebleu, as a naive regular expression would produce. The designation and cleaned description are then merged into a single text field, which is lowercased and accent-folded to reduce vocabulary fragmentation. Finally, a binary desc_missing indicator is built from the original column, before any fill.

4.2 Train/test split

The data are split 80/20 into training and test sets, stratified on prdtypecode so that both sets preserve the 27-class proportions. The test set is held out purely for evaluation. Any transformation that learns from the corpus is fitted on the training set only, to avoid data leakage.

4.3 TF-IDF vectorization

TF-IDF (implemented with scikit-learn [8]) converts each listing's merged text into a numeric vector [6]. For each term, the term frequency (how often it occurs in the listing) is multiplied by the inverse document frequency, defined as the logarithm of the number of documents divided by the number of documents containing the term. Ubiquitous words such as de, le and et therefore receive near-zero weight, while rare, discriminative words such as nintendo switch receive high weight. Both unigrams (single words) and bigrams (adjacent word-pairs) are used, because some category signal lives only in the pair: jeux video (video games) is a strong marker that neither jeux nor video conveys alone. Bigrams greatly enlarge the vocabulary, so it is capped at the 20,000 most useful terms out of approximately 280,000.

The result is a sparse matrix whose rows are the listings and whose columns are the 20,000 vocabulary terms, with each cell a TF-IDF score; approximately 99% of the entries are zero. The vectorizer is fitted on the training set only and then applied to transform both the training and test sets, producing two sparse matrices that share the same vocabulary. The deliverable handed to the modelling stage is therefore a numeric feature set of approximately 67,900 training and 17,000 test rows, comprising the 20,000 sparse TF-IDF features together with the desc_missing and length features, over the 27 target categories.

References

1. Amoualian, H., Goswami, P., Das, P., Montalvo, P., Ach, L., & Dean, N. R. (2021). An E-Commerce Dataset in French for Multi-modal Product Categorization and Cross-Modal Retrieval. In Advances in Information Retrieval (ECIR 2021), LNCS 12657, 18-31. Springer. DOI: 10.1007/978-3-030-72113-8_2.

2. Little, R. J. A., & Rubin, D. B. (2019). Statistical Analysis with Missing Data (3rd ed.). Wiley. DOI: 10.1002/9781119482260.

3. Cramer, H. (1946). Mathematical Methods of Statistics (Princeton Mathematical Series, vol. 9). Princeton University Press.

4. Kruskal, W. H., & Wallis, W. A. (1952). Use of ranks in one-criterion variance analysis. Journal of the American Statistical Association, 47(260), 583-621. DOI: 10.1080/01621459.1952.10483441.

5. Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. Journal of the American Statistical Association, 22(158), 209-212. DOI: 10.1080/01621459.1927.10502953.

6. Sparck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. Journal of Documentation, 28(1), 11-21. DOI: 10.1108/eb026526.

7. Richardson, L. (2007). Beautiful Soup [Python library, HTML/XML parser]. https://www.crummy.com/software/BeautifulSoup/.

8. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.